DO NOW — RECALL

Programming Techniques · Modular Design and IDE Deb

Name: _____ Class: _____ Date: _____

1. What does ref do in C#?

2. What is variable scope?

3. What does a void method return?

4. State one advantage of a hash table.

DO NOW — CHECK / OBJECTIVES

Correct and improve

Check page 1 in GREEN PEN. Write corrections in full.

1. Model answer

Allows access to and modification of caller variable

2. Model answer

Where a variable is accessible

3. Model answer

No value

4. Model answer

Fast average access when well designed

By the end, you can…

• Decompose a program into methods

• Use breakpoints and variable inspection

• Correct a modular C# bug

Vocabulary — know these terms

Recall the term — modularity

Recall the term — breakpoint

Recall the term — step into

Recall the term — watch

Recall the term — testability

WE DO — SOURCE CHECK / IMPROVE

Only use this section AFTER attempting the We Do, You Do and Exit Ticket. Compare with the original classroom lesson's CHECK explanation. Correct errors in GREEN PEN and explain why.

Guided 1 — original CHECK

Check your thinking

Not all code paths return a value.

IMPROVE: Explain one correction to your own reasoning.

Guided 2 — original CHECK

Discuss, then respond

Why step into a function in a debugger?

Check your thinking

To inspect statements inside the called method.

IMPROVE: Explain one correction to your own reasoning.

EXIT TICKET / REFLECTION

Check your understanding

Complete the actual lesson Exit Ticket before checking the answer.

True or false: modular code can be easier to test independently.

How confident are you? Circle one: Confident / Getting there / Need help

Explain your choice, identifying one concept you can now use independently.

GREEN PEN — Correct and explain

Choose one mistake from your independent work. Write the correct method and explain precisely why your original answer was wrong.

Next step: revisit the model or a relevant revision-guide section if needed.

NEW LEARNING

Read, annotate and explain

Trace a method call

static int Sum(int[] marks) { int total = 0; for (int i=0; i<=marks.Length; i++) total += marks[i]; return total; }

Explain the important decision

Break on the array access, Step Into Sum and watch i, marks.Length and total. Diagnose the failing state before editing the loop condition.

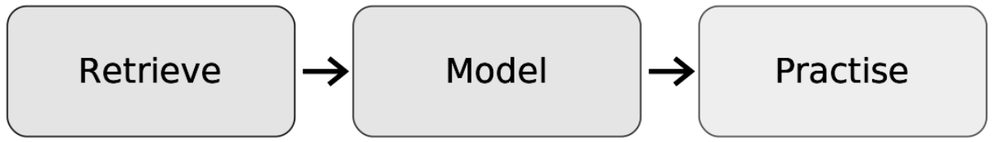
Explain one step in your own words, using the correct technical terms.

New Learning · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

Trace a method call

static int Sum(int[] marks) { int total = 0; for (int i=0; i<=marks.Length; i++) total += marks[i]; return total; }



Three-stage diagram showing retrieve, model, and practise

YOU DO — INDEPENDENT PRACTICE

Questions · Part 2

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

Question 3

Apply independently
Correct: static int Add(int a,int b){ Console.WriteLine(a+b); } [2]

YOU DO — INDEPENDENT PRACTICE

Questions · Part 1

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

Question 1

Apply independently
Give two benefits of modular design. [2]

Question 2

Apply independently
Describe how a breakpoint helps debug a loop. [2]

I DO — WORKED EXAMPLE

Follow the worked method

Trace a method call

Pause and inspect

Call Sum with [4,6]. Step Into enters the method; Step Over a call would run it without following its internal statements. Watch total=0, Length=2 and i=0.

Locate the first invalid access

At i=0 total becomes 4; at i=1 it becomes 10. The faulty test allows i=2, but valid indexes are 0 and 1. The next array access fails.

Correct and retest

Change i<=marks.Length to i<marks.Length. Retest [4,6]→10, []→0 and [7]→7. A small routine can now be tested independently and reused.

WE DO — GUIDED PRACTICE

Try → Discuss → Improve

1. Problem 1

A method has no return despite return type int. What is wrong?

Method / working:

2. Problem 2

Why step into a function in a debugger?

Method / working:

WE DO — TRY · DISCUSS · IMPROVE

TRY: Use the original guided questions and diagrams in the preceding We Do pages. DISCUSS: Explain and challenge methods before independent practice. IMPROVE: Note one change now; compare with the source model only AFTER the Exit Ticket.

Guided 1: Problem 1 A method has no return despite return type int. What is wrong? Discuss, then respond

DISCUSS — Compare your methods. Justify one important decision, identify one difference, and agree a specific improvement.

IMPROVE — Write one precise correction or stronger explanation in your own answer.

Guided 2: Problem 2 Why step into a function in a debugger?

DISCUSS — Compare approaches, walk through one test input, and explain how you would detect or correct a mistake.

IMPROVE — Write one precise correction or stronger explanation in your own answer.